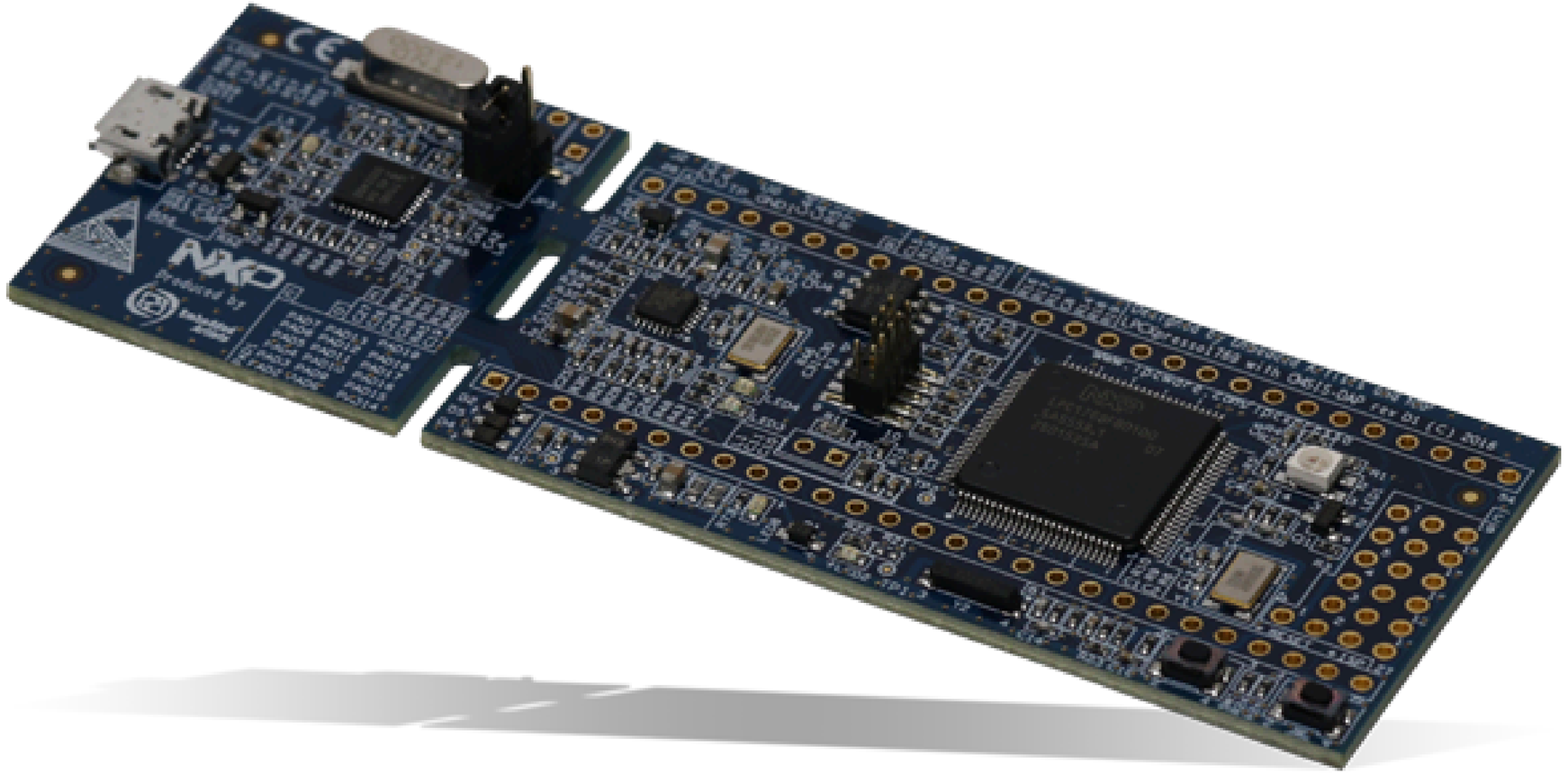
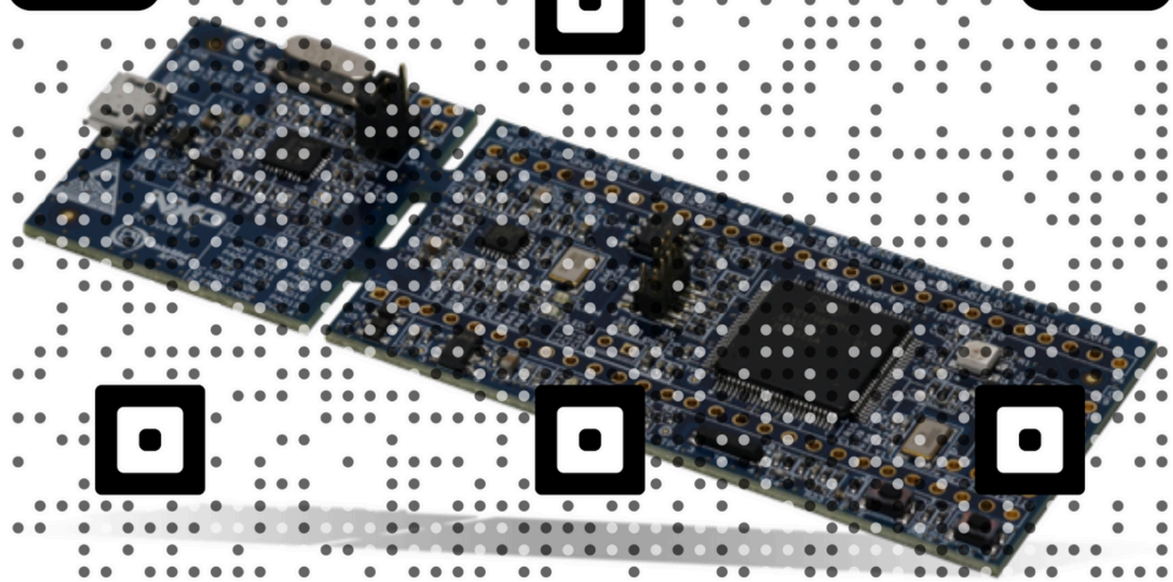


TIMERS

David Trujillo



Timers

David Trujillo

AVAILABLE PINS

TIMER 0

CAP 0.0: P1.26 (11)
CAP 0.1: P1.27 (11)
MAT 0.0: P1.28 (11), P3.25 (10)
MAT 0.1: P1.29 (11), P3.26 (10)

TIMER 1

CAP 1.0: P1.18 (11)
CAP 1.1: P1.19 (11)
MAT 1.0: P1.22 (11)
MAT 1.1: P1.25 (11)

TIMER 2

CAP 2.0: P0.4 (11)
CAP 2.1: P0.5 (11)
MAT 2.0: P0.6 (11), P4.28 (10)
MAT 2.1: P0.7 (11), P4.29 (10)
MAT 2.2: P0.8 (11)
MAT 2.3: P0.9 (11)

TIMER 3

CAP 3.0: P0.23 (11)
CAP 3.1: P0.24 (11)
MAT 3.0: P0.10 (11)
MAT 3.1: P0.11 (11)

David Trujillo

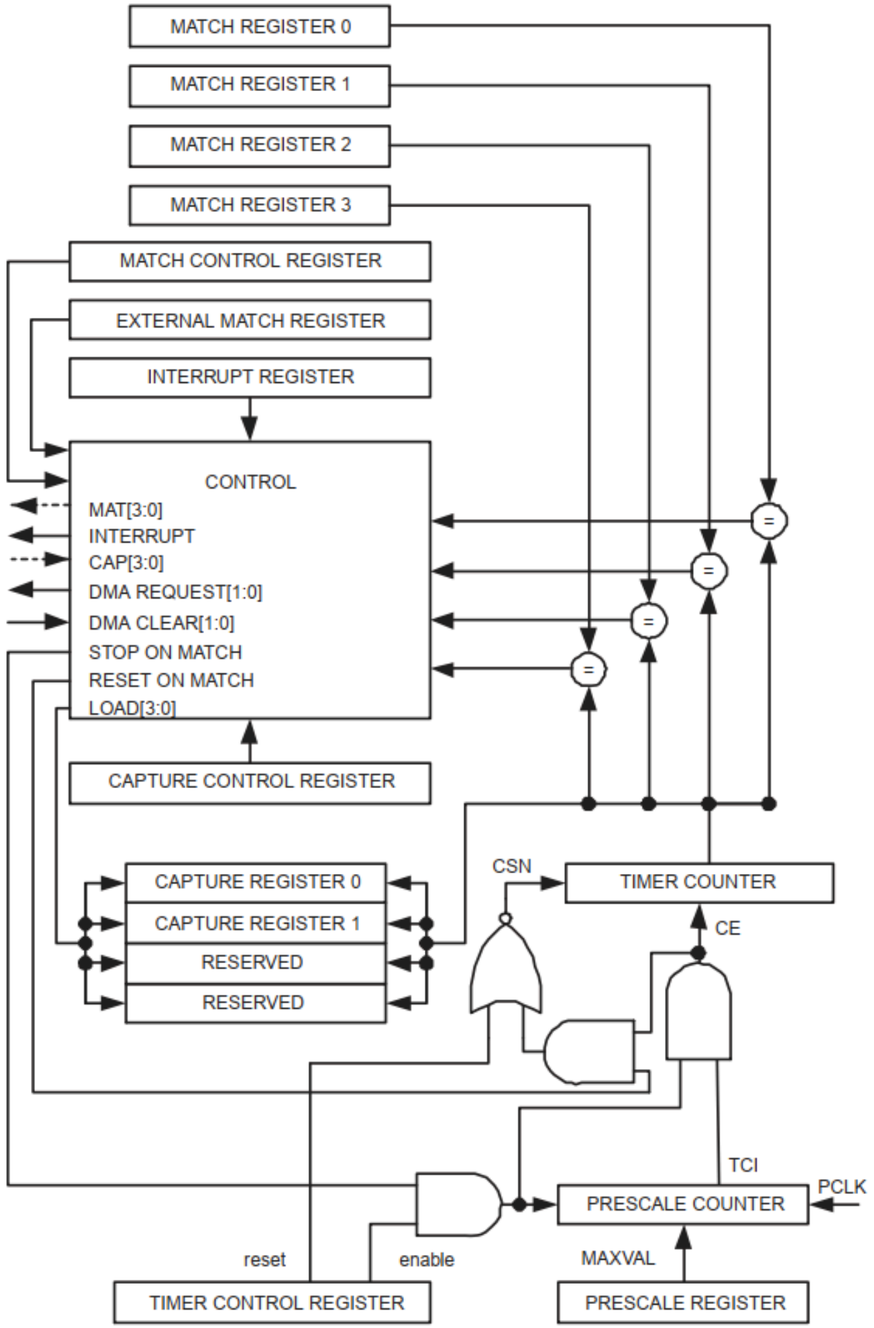


Fig 117. Timer block diagram

CLOCK CONTROL

David Trujillo

Table 46. Power Control for Peripherals register (PCONP - address 0x400F C0C4) bit description

Bit	Symbol	Description	Reset value
0	-	Reserved.	NA
1	PCTIM0	Timer/Counter 0 power/clock control bit.	1
2	PCTIM1	Timer/Counter 1 power/clock control bit.	1
		⋮	
22	PCTIM2	Timer 2 power/clock control bit.	0
23	PCTIM3	Timer 3 power/clock control bit.	0

Table 40. Peripheral Clock Selection register 0 (PCLKSEL0 - address 0x400F C1A8) bit description

Bit	Symbol	Description	Reset value
1:0	PCLK_WDT	Peripheral clock selection for WDT.	00
3:2	PCLK_TIMER0	Peripheral clock selection for TIMER0.	00
5:4	PCLK_TIMER1	Peripheral clock selection for TIMER1.	00

Table 41. Peripheral Clock Selection register 1 (PCLKSEL1 - address 0x400F C1AC) bit description

Bit	Symbol	Description	Reset value
13:12	PCLK_TIMER2	Peripheral clock selection for TIMER2.	00
15:14	PCLK_TIMER3	Peripheral clock selection for TIMER3.	00

Table 42. Peripheral Clock Selection register bit values

PCLKSEL0 and PCLKSEL1 individual peripheral's clock select options	Function	Reset value
00	PCLK_peripheral = CCLK/4	00
01	PCLK_peripheral = CCLK	
10	PCLK_peripheral = CCLK/2	
11	PCLK_peripheral = CCLK/8, except for CAN1, CAN2, and CAN filtering when "11" selects = CCLK/6.	

```
typedef struct
{
  __IO uint32_t FLASHCFG;
       uint32_t RESERVED0[31];
  __IO uint32_t PLL0CON;
  __IO uint32_t PLL0CFG;
  __I  uint32_t PLL0STAT;
  __O  uint32_t PLL0FEED;
       uint32_t RESERVED1[4];
  __IO uint32_t PLL1CON;
  __IO uint32_t PLL1CFG;
  __I  uint32_t PLL1STAT;
  __O  uint32_t PLL1FEED;
       uint32_t RESERVED2[4];
  __IO uint32_t PCON;
  __IO uint32_t PCONP;
       uint32_t RESERVED3[15];
  __IO uint32_t CCLKCFG;
  __IO uint32_t USBCLKCFG;
  __IO uint32_t CLKSRCSEL;
  __IO uint32_t CANSLEEPCLR;
  __IO uint32_t CANWAKEFLAGS;
       uint32_t RESERVED4[10];
  __IO uint32_t EXTINT;
       uint32_t RESERVED5;
  __IO uint32_t EXTMODE;
  __IO uint32_t EXTPOLAR;
       uint32_t RESERVED6[12];
  __IO uint32_t RSID;
       uint32_t RESERVED7[7];
  __IO uint32_t SCS;
  __IO uint32_t IRCTRIM;
  __IO uint32_t PCLKSEL0;
  __IO uint32_t PCLKSEL1;
       uint32_t RESERVED8[4];
  __IO uint32_t USBIntSt;
  __IO uint32_t DMAREQSEL;
  __IO uint32_t CLKOUTCFG;
} LPC_SC_TypeDef;
```

LPC_SC →

SystemCoreClock

David Trujillo

Table 426. TIMER/COUNTER0-3 register map

Generic Name	Description	Access	Reset Value[1]	TIMERn Register/ Name & Address
IR	Interrupt Register. The IR can be written to clear interrupts. The IR can be read to identify which of eight possible interrupt sources are pending.	R/W	0	T0IR - 0x4000 4000 T1IR - 0x4000 8000 T2IR - 0x4009 0000 T3IR - 0x4009 4000
TCR	Timer Control Register. The TCR is used to control the Timer Counter functions. The Timer Counter can be disabled or reset through the TCR.	R/W	0	T0TCR - 0x4000 4004 T1TCR - 0x4000 8004 T2TCR - 0x4009 0004 T3TCR - 0x4009 4004
TC	Timer Counter. The 32-bit TC is incremented every PR+1 cycles of PCLK. The TC is controlled through the TCR.	R/W	0	T0TC - 0x4000 4008 T1TC - 0x4000 8008 T2TC - 0x4009 0008 T3TC - 0x4009 4008
PR	Prescale Register. When the Prescale Counter (below) is equal to this value, the next clock increments the TC and clears the PC.	R/W	0	T0PR - 0x4000 400C T1PR - 0x4000 800C T2PR - 0x4009 000C T3PR - 0x4009 400C
PC	Prescale Counter. The 32-bit PC is a counter which is incremented to the value stored in PR. When the value in PR is reached, the TC is incremented and the PC is cleared. The PC is observable and controllable through the bus interface.	R/W	0	T0PC - 0x4000 4010 T1PC - 0x4000 8010 T2PC - 0x4009 0010 T3PC - 0x4009 4010

Table 426. TIMER/COUNTER0-3 register map

Generic Name	Description	Access	Reset Value[1]	TIMERn Register/ Name & Address
MCR	Match Control Register. The MCR is used to control if an interrupt is generated and if the TC is reset when a Match occurs.	R/W	0	T0MCR - 0x4000 4014 T1MCR - 0x4000 8014 T2MCR - 0x4009 0014 T3MCR - 0x4009 4014
MR0	Match Register 0. MR0 can be enabled through the MCR to reset the TC, stop both the TC and PC, and/or generate an interrupt every time MR0 matches the TC.	R/W	0	T0MR0 - 0x4000 4018 T1MR0 - 0x4000 8018 T2MR0 - 0x4009 0018 T3MR0 - 0x4009 4018
MR1	Match Register 1. See MR0 description.	R/W	0	T0MR1 - 0x4000 401C T1MR1 - 0x4000 801C T2MR1 - 0x4009 001C T3MR1 - 0x4009 401C
MR2	Match Register 2. See MR0 description.	R/W	0	T0MR2 - 0x4000 4020 T1MR2 - 0x4000 8020 T2MR2 - 0x4009 0020 T3MR2 - 0x4009 4020
MR3	Match Register 3. See MR0 description.	R/W	0	T0MR3 - 0x4000 4024 T1MR3 - 0x4000 8024 T2MR3 - 0x4009 0024 T3MR3 - 0x4009 4024

Table 426. TIMER/COUNTER0-3 register map

Generic Name	Description	Access	Reset Value[1]	TIMERn Register/ Name & Address
CCR	Capture Control Register. The CCR controls which edges of the capture inputs are used to load the Capture Registers and whether or not an interrupt is generated when a capture takes place.	R/W	0	T0CCR - 0x4000 4028 T1CCR - 0x4000 8028 T2CCR - 0x4009 0028 T3CCR - 0x4009 4028
CR0	Capture Register 0. CR0 is loaded with the value of TC when there is an event on the CAPn.0(CAP0.0 or CAP1.0 respectively) input.	RO	0	T0CR0 - 0x4000 402C T1CR0 - 0x4000 802C T2CR0 - 0x4009 002C T3CR0 - 0x4009 402C
CR1	Capture Register 1. See CR0 description.	RO	0	T0CR1 - 0x4000 4030 T1CR1 - 0x4000 8030 T2CR1 - 0x4009 0030 T3CR1 - 0x4009 4030
EMR	External Match Register. The EMR controls the external match pins MATn.0-3 (MAT0.0-3 and MAT1.0-3 respectively).	R/W	0	T0EMR - 0x4000 403C T1EMR - 0x4000 803C T2EMR - 0x4009 003C T3EMR - 0x4009 403C
CTCR	Count Control Register. The CTCR selects between Timer and Counter mode, and in Counter mode selects the signal and edge(s) for counting.	R/W	0	T0CTCR - 0x4000 4070 T1CTCR - 0x4000 8070 T2CTCR - 0x4009 0070 T3CTCR - 0x4009 4070

INTERRUPT REGISTER

Table 427. Interrupt Register (T[0/1/2/3]IR - addresses 0x4000 4000, 0x4000 8000, 0x4009 0000, 0x4009 4000) bit description

Bit	Symbol	Description	Reset Value
0	MR0 Interrupt	Interrupt flag for match channel 0.	0
1	MR1 Interrupt	Interrupt flag for match channel 1.	0
2	MR2 Interrupt	Interrupt flag for match channel 2.	0
3	MR3 Interrupt	Interrupt flag for match channel 3.	0
4	CR0 Interrupt	Interrupt flag for capture channel 0 event.	0
5	CR1 Interrupt	Interrupt flag for capture channel 1 event.	0
31:6	-	Reserved	-

David Trujillo

TIMER CONTROL REGISTER

Table 428. Timer Control Register (TCR, TIMERn: TnTCR - addresses 0x4000 4004, 0x4000 8004, 0x4009 0004, 0x4009 4004) bit description

Bit	Symbol	Description	Reset Value
0	Counter Enable	When one, the Timer Counter and Prescale Counter are enabled for counting. When zero, the counters are disabled.	0
1	Counter Reset	When one, the Timer Counter and the Prescale Counter are synchronously reset on the next positive edge of PCLK. The counters remain reset until TCR[1] is returned to zero.	0
31:2	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

David Trujillo

COUNT CONTROL REGISTER

Table 429. Count Control Register (T[0/1/2/3]CTCR - addresses 0x4000 4070, 0x4000 8070, 0x4009 0070, 0x4009 4070) bit description

Bit	Symbol	Value	Description	Reset Value
1:0	Counter/ Timer Mode		This field selects which rising PCLK edges can increment the Timer's Prescale Counter (PC), or clear the PC and increment the Timer Counter (TC).	00
		00	Timer Mode: the TC is incremented when the Prescale Counter matches the Prescale Register. The Prescale Counter is incremented on every rising PCLK edge.	
		01	Counter Mode: TC is incremented on rising edges on the CAP input selected by bits 3:2.	
		10	Counter Mode: TC is incremented on falling edges on the CAP input selected by bits 3:2.	
		11	Counter Mode: TC is incremented on both edges on the CAP input selected by bits 3:2.	
3:2	Count Input Select		When bits 1:0 in this register are not 00, these bits select which CAP pin is sampled for clocking.	00
		00	CAPn.0 for TIMERn	
		01	CAPn.1 for TIMERn	
		10	Reserved	
		11	Reserved	
			Note: If Counter mode is selected for a particular CAPn input in the TnCTCR, the 3 bits for that input in the Capture Control Register (TnCCR) must be programmed as 000. However, capture and/or interrupt can be selected for the other 3 CAPn inputs in the same timer.	
31:4	-	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

David Trujillo

TIMER COUNTER

21.6.4 Timer Counter registers (T0TC - T3TC, 0x4000 4008, 0x4000 8008, 0x4009 0008, 0x4009 4008)

The 32-bit Timer Counter register is incremented when the prescale counter reaches its terminal count. Unless it is reset before reaching its upper limit, the Timer Counter will count up through the value 0xFFFF FFFF and then wrap back to the value 0x0000 0000. This event does not cause an interrupt, but a match register can be used to detect an overflow if needed.

David Trujillo

21.6.5 Prescale register (T0PR - T3PR, 0x4000 400C, 0x4000 800C, 0x4009 000C, 0x4009 400C)

The 32-bit Prescale register specifies the maximum value for the Prescale Counter

21.6.6 Prescale Counter register (T0PC - T3PC, 0x4000 4010, 0x4000 8010, 0x4009 0010, 0x4009 4010)

The 32-bit Prescale Counter controls division of PCLK by some constant value before it is applied to the Timer Counter. This allows control of the relationship of the resolution of the timer versus the maximum time before the timer overflows. The Prescale Counter is incremented on every PCLK. When it reaches the value stored in the Prescale register, the Timer Counter is incremented and the Prescale Counter is reset on the next PCLK. This causes the Timer Counter to increment on every PCLK when PR = 0, every 2 pclks when PR = 1, etc.

21.6.7 Match Registers (MR0 - MR3)

The Match register values are continuously compared to the Timer Counter value. When the two values are equal, actions can be triggered automatically. The action possibilities are to generate an interrupt, reset the Timer Counter, or stop the timer. Actions are controlled by the settings in the MCR register.

21.6.9 Capture Registers (CR0 - CR1)

Each Capture register is associated with a device pin and may be loaded with the Timer Counter value when a specified event occurs on that pin. The settings in the Capture Control Register register determine whether the capture function is enabled, and whether a capture event happens on the rising edge of the associated pin, the falling edge, or on both edges.

MATCH CONTROL

Table 430. Match Control Register (T[0/1/2/3]MCR - addresses 0x4000 4014, 0x4000 8014, 0x4009 0014, 0x4009 4014) bit description

Bit	Symbol	Value	Description	Reset Value
0	MR0I	1	Interrupt on MR0: an interrupt is generated when MR0 matches the value in the TC.	0
		0	This interrupt is disabled	
1	MR0R	1	Reset on MR0: the TC will be reset if MR0 matches it.	0
		0	Feature disabled.	
2	MR0S	1	Stop on MR0: the TC and PC will be stopped and TCR[0] will be set to 0 if MR0 matches the TC.	0
		0	Feature disabled.	
3	MR1I	1	Interrupt on MR1: an interrupt is generated when MR1 matches the value in the TC.	0
		0	This interrupt is disabled	
4	MR1R	1	Reset on MR1: the TC will be reset if MR1 matches it.	0
		0	Feature disabled.	
5	MR1S	1	Stop on MR1: the TC and PC will be stopped and TCR[0] will be set to 0 if MR1 matches the TC.	0
		0	Feature disabled.	

David Trujillo

CAPTURE CONTROL

Table 431. Capture Control Register (T[0/1/2/3]CCR - addresses 0x4000 4028, 0x4000 8020, 0x4009 0028, 0x4009 4028) bit description

Bit	Symbol	Value	Description	Reset Value
0	CAP0RE	1	Capture on CAPn.0 rising edge: a sequence of 0 then 1 on CAPn.0 will cause CR0 to be loaded with the contents of TC.	0
		0	This feature is disabled.	
1	CAP0FE	1	Capture on CAPn.0 falling edge: a sequence of 1 then 0 on CAPn.0 will cause CR0 to be loaded with the contents of TC.	0
		0	This feature is disabled.	
2	CAP0I	1	Interrupt on CAPn.0 event: a CR0 load due to a CAPn.0 event will generate an interrupt.	0
		0	This feature is disabled.	
3	CAP1RE	1	Capture on CAPn.1 rising edge: a sequence of 0 then 1 on CAPn.1 will cause CR1 to be loaded with the contents of TC.	0
		0	This feature is disabled.	
4	CAP1FE	1	Capture on CAPn.1 falling edge: a sequence of 1 then 0 on CAPn.1 will cause CR1 to be loaded with the contents of TC.	0
		0	This feature is disabled.	
5	CAP1I	1	Interrupt on CAPn.1 event: a CR1 load due to a CAPn.1 event will generate an interrupt.	0
		0	This feature is disabled.	
31:6	-		Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

David Trujillo

EXTERNAL MATCH

Table 432. External Match Register (T[0/1/2/3]EMR - addresses 0x4000 403C, 0x4000 803C, 0x4009 003C, 0x4009 403C) bit description

Bit	Symbol	Description	Reset Value
0	EM0	External Match 0. When a match occurs between the TC and MR0, this bit can either toggle, go low, go high, or do nothing, depending on bits 5:4 of this register. This bit can be driven onto a MATn.0 pin in a positive-logic manner (0 = low, 1 = high).	0
1	EM1	External Match 1. When a match occurs between the TC and MR1, this bit can either toggle, go low, go high, or do nothing, depending on bits 7:6 of this register. This bit can be driven onto a MATn.1 pin, in a positive-logic manner (0 = low, 1 = high).	0
2	EM2	External Match 2. When a match occurs between the TC and MR2, this bit can either toggle, go low, go high, or do nothing, depending on bits 9:8 of this register. This bit can be driven onto a MATn.2 pin, in a positive-logic manner (0 = low, 1 = high).	0
3	EM3	External Match 3. When a match occurs between the TC and MR3, this bit can either toggle, go low, go high, or do nothing, depending on bits 11:10 of this register. This bit can be driven onto a MATn.3 pin, in a positive-logic manner (0 = low, 1 = high).	0
5:4	EMC0	External Match Control 0. Determines the functionality of External Match 0. Table 433 shows the encoding of these bits.	00
7:6	EMC1	External Match Control 1. Determines the functionality of External Match 1. Table 433 shows the encoding of these bits.	00
9:8	EMC2	External Match Control 2. Determines the functionality of External Match 2. Table 433 shows the encoding of these bits.	00
11:10	EMC3	External Match Control 3. Determines the functionality of External Match 3. Table 433 shows the encoding of these bits.	00
15:12	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

David Trujillo

Table 433. External Match Control

EMR[11:10], EMR[9:8], EMR[7:6], or EMR[5:4]	Function
00	Do Nothing.
01	Clear the corresponding External Match bit/output to 0 (MATn.m pin is LOW if pinned out).
10	Set the corresponding External Match bit/output to 1 (MATn.m pin is HIGH if pinned out).
11	Toggle the corresponding External Match bit/output.

```
typedef struct
{
  __IO uint32_t IR;
  __IO uint32_t TCR;
  __IO uint32_t TC;
  __IO uint32_t PR;
  __IO uint32_t PC;
  __IO uint32_t MCR;
  __IO uint32_t MR0;
  __IO uint32_t MR1;
  __IO uint32_t MR2;
  __IO uint32_t MR3;
  __IO uint32_t CCR;
  __I  uint32_t CR0;
  __I  uint32_t CR1;
       uint32_t RESERVED0[2];
  __IO uint32_t EMR;
       uint32_t RESERVED1[12];
  __IO uint32_t CTCR;
} LPC_TIM_TypeDef;
```

LPC_TIMx →

David Trujillo

```
#define LPC_TIM0                ((LPC_TIM_TypeDef       *) LPC_TIM0_BASE     )
#define LPC_TIM1                ((LPC_TIM_TypeDef       *) LPC_TIM1_BASE     )
#define LPC_TIM2                ((LPC_TIM_TypeDef       *) LPC_TIM2_BASE     )
#define LPC_TIM3                ((LPC_TIM_TypeDef       *) LPC_TIM3_BASE     )
```

EXAMPLES

David Trujillo

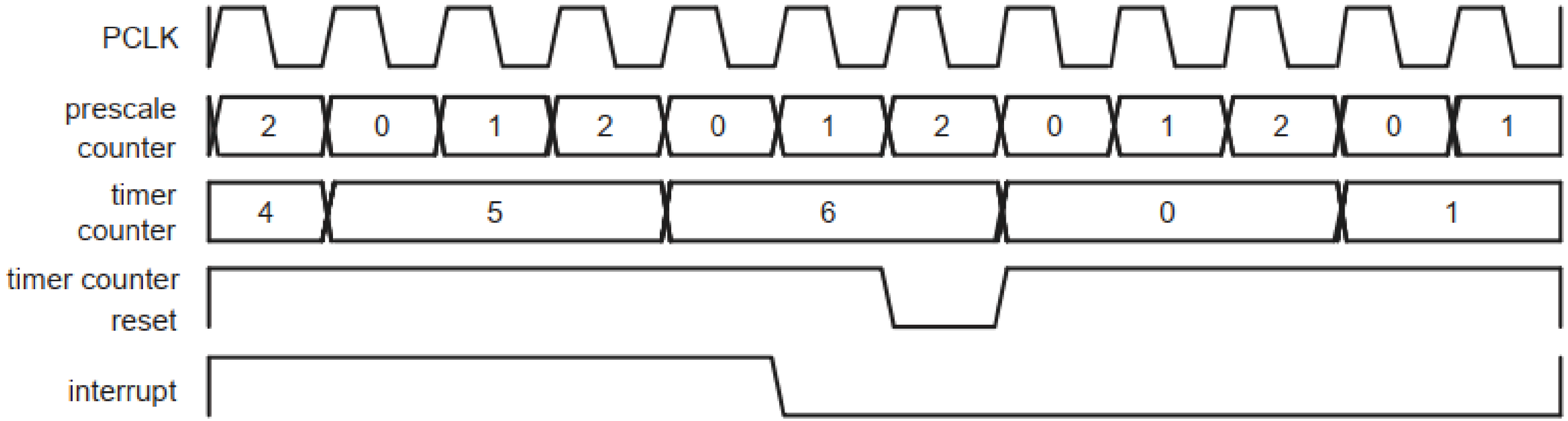


Fig 115. A timer cycle in which PR=2, MRx=6, and both interrupt and reset on match are enabled.

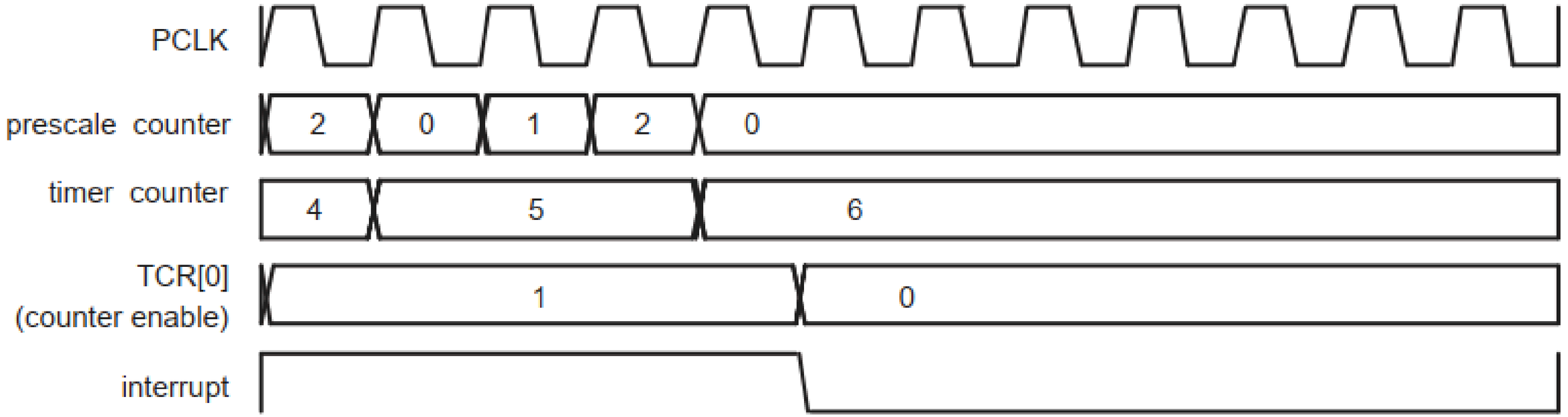


Fig 116. A timer Cycle in Which PR=2, MRx=6, and both interrupt and stop on match are enabled

TIME CALCULATION

David Trujillo

Assuming:

Timer clock = ¼ SystemCoreClock

25 [MHz] as default

David Trujillo

$$f_{TC} = f_{clock}/(PR+1)$$

$$TC_{count} = (PR+1)/f_{clock}$$

$$1\ [ms] = (PR+1)/(25\ [MHz])$$

$$PR = 24999$$

David Trujillo

$$T_{int} = (MR+1)(PR+1)/f_{clock}$$

$$MR = (T_{int} \cdot f_{clock}/(PR+1))-1$$

$$T_{int} = 50\ [ms] = (MR+1) \cdot (1\ [ms])$$

David Trujillo

DRIVERS

David Trujillo

```
typedef enum {
    TIM_MR0_INT = 0,
    TIM_MR1_INT,
    TIM_MR2_INT,
    TIM_MR3_INT,
    TIM_CR0_INT,
    TIM_CR1_INT
} TIM_INT_TYPE;
```

```
typedef enum {
    TIM_TICKVAL = 0,
    TIM_USVAL
} TIM_PRESCALE_OPT;
```

```
typedef enum {
    TIM_CAPTURE_CHANNEL_0 = 0,
    TIM_CAPTURE_CHANNEL_1
} TIM_CAPTURE_CHANNEL_OPT;
```

```
typedef enum {
    TIM_NOTHING = 0,
    TIM_LOW,
    TIM_HIGH,
    TIM_TOGGLE
} TIM_EXTMATCH_OPT;
```

```
typedef enum {
    TIM_NONE = 0,
    TIM_RISING,
    TIM_FALLING,
    TIM_ANY
} TIM_CAP_MODE_OPT;
```

```
typedef enum {
    TIM_TIMER_MODE = 0,
    TIM_COUNTER_RISING_MODE,
    TIM_COUNTER_FALLING_MODE,
    TIM_COUNTER_ANY_MODE
} TIM_MODE_OPT;
```

```
typedef struct {
    TIM_PRESCALE_OPT    prescaleOption;
    uint32_t            prescaleValue;
} TIM_TIMERCFG_Type;
```

- TIM_TICKVAL : Absolute value.
- TIM_USVAL : Value in microseconds.

Prescale max value.

```
typedef struct {
    TIM_CAPTURE_CHANNEL_OPT countInputSelect;
} TIM_COUNTERCFG_Type;
```

- TIM_CAPTURE_CHANNEL_0 : CAPn.0 input pin for TIMERn.
- TIM_CAPTURE_CHANNEL_1 : CAPn.1 input pin for TIMERn.

```
typedef struct {
    TIM_MATCH_CHANNEL_OPT   matchChannel;
    FunctionalState         intOnMatch;

    FunctionalState         stopOnMatch;

    FunctionalState         resetOnMatch;

    TIM_EXTMATCH_OPT        extMatchOutputType;

    uint32_t                matchValue;
} TIM_MATCHCFG_Type;
```

TIM_MATCH_CHANNEL_x [0...3].

- ENABLE : Enable interrupt on match.
- DISABLE : Disable interrupt on match.
- ENABLE : Stop timer on match.
- DISABLE : Do not stop timer on match.
- ENABLE : Reset timer on match.
- DISABLE : Do not reset timer on match.

- TIM_NOTHING : Do nothing for external output pin if matched.
- TIM_LOW : Force external output pin to low if matched.
- TIM_HIGH : Force external output pin to high if matched.
- TIM_TOGGLE : Toggle external output pin if matched.

Match value to compare with timer counter.

```
typedef struct {
    TIM_CAPTURE_CHANNEL_OPT captureChannel;
    FunctionalState         risingEdge;


    FunctionalState         fallingEdge;


    FunctionalState         intOnCapture;

} TIM_CAPTURECFG_Type;
```

TIM_CAPTURE_CHANNEL_x [0...1].

- ENABLE : Enable capture on rising edge.
- DISABLE : Disable capture on rising edge.
- ENABLE : Enable capture on falling edge.
- DISABLE : Disable capture on falling edge.
- ENABLE : Enable interrupt on capture event.
- DISABLE : Disable interrupt on capture event.

```
void TIM_Init(
    LPC_TIM_TypeDef *TIMx,
    TIM_MODE_OPT timerCounterMode,
    void *TIM_ConfigStruct)
```

Initializes the specified Timer/Counter peripheral.

This function enables the power and clock for the selected timer, configures its mode (timer or counter), sets the prescaler or counter input as required, resets the Timer Counter (TC) and Prescale Counter (PC), and clears all pending interrupt flags. It prepares the timer for further configuration and use.

Params: *[in]* **TIMx** — Pointer to the timer peripheral (LPC_TIMx [0...3]).
[in] **timerCounterMode** — Timer/counter mode selection:

- TIM_TIMER_MODE
- TIM_COUNTER_RISING_MODE
- TIM_COUNTER_FALLING_MODE
- TIM_COUNTER_ANY_MODE

[in] **TIM_ConfigStruct** — Pointer to configuration structure:

- TIM_TIMERCFG_Type for timer mode
- TIM_COUNTERCFG_Type for counter mode

Remarks:

- The function enables the timer's power and sets the peripheral clock divider.
- It resets and initializes the prescaler and counters.
- It clears all interrupt flags in the IR register.
- The timer is left in a disabled state after initialization.

H `lpc17xx_timer.h`

```
void TIM_DeInit(LPC_TIM_TypeDef *TIMx)
```

De-initializes the specified Timer/Counter peripheral.

This function disables the timer, and removes power from the selected timer peripheral. It should be called to safely power down the timer and release its resources.

Params: *[in]* **TIMx** — Pointer to the timer peripheral (LPC_TIMx [0...3]).

Remarks:

- The function disables the timer.
- It disables the peripheral clock and powers down the timer.
- After calling this function, the timer must be re-initialized before use.

H `lpc17xx_timer.h`

```
void TIM_ClearIntPending(
    LPC_TIM_TypeDef* TIMx,
    TIM_INT_TYPE intFlag)
```

Clears the specified Timer/Counter interrupt pending flag.

This function clears the interrupt pending flag for the given match or capture channel in the timer's interrupt register (IR). It can be used for both match and capture interrupts.

Params: *[in]* **TIMx** — Pointer to the timer peripheral (LPC_TIMx [0...3]).
[in] **intFlag** — Interrupt type to clear:

- TIM_MRx_INT: Match channel x [0..3]
- TIM_CRx_INT: Capture channel x [0..1]

H `lpc17xx_timer.h`

```
FlagStatus TIM_GetIntStatus(
    LPC_TIM_TypeDef* TIMx,
    TIM_INT_TYPE intFlag)
```

Gets the interrupt status for the specified Timer/Counter channel.

This function checks if the interrupt flag for the given match or capture channel is set in the timer's interrupt register (IR). It can be used for both match and capture interrupts.

Params: *[in]* **TIMx** — Pointer to the timer peripheral (LPC_TIMx [0...3]).
[in] **intFlag** — Interrupt type to check:

- TIM_MRx_INT: Match channel x [0..3]
- TIM_CRx_INT: Capture channel x [0..1]

Returns: FlagStatus

- SET : Interrupt is pending
- RESET : No interrupt pending

H `lpc17xx_timer.h`

```
void TIM_ConfigStructInit(
    TIM_MODE_OPT timerCounterMode,
    void* TIM_ConfigStruct)
```

Initializes a timer or counter configuration structure with default values.

This function sets default values for the provided configuration structure, depending on the selected mode. For timer mode, it sets the prescale option to microseconds and the prescale value to 0. For counter mode, it sets the count input select to CAPn.0. Reserved fields are not initialized.

Params: *[in]* **timerCounterMode** — Timer/counter mode selection:

- TIM_TIMER_MODE
- TIM_COUNTER_RISING_MODE
- TIM_COUNTER_FALLING_MODE
- TIM_COUNTER_ANY_MODE

[out] **TIM_ConfigStruct** — Pointer to configuration structure to initialize:

- TIM_TIMERCFG_Type for timer mode
- TIM_COUNTERCFG_Type for counter mode

Remarks: Call this function before configuring a timer or counter to ensure the structure has valid default values.

H `lpc17xx_timer.h`

```
void TIM_ConfigMatch(
    LPC_TIM_TypeDef* TIMx,
    TIM_MATCHCFG_Type* TIM_MatchConfigStruct)
```

Configures the match channel for the specified Timer/Counter peripheral.

This function sets up the match value, interrupt, reset, stop, and external match output for the selected match channel. It also clears the corresponding interrupt flag before configuration to avoid spurious interrupts.

Params: *[in]* **TIMx** — Pointer to the timer peripheral (LPC_TIMx [0...3]).
[in] **TIM_MatchConfigStruct** — Pointer to a TIM_MATCHCFG_Type structure.

Remarks:

- The interrupt flag for the selected channel is cleared before configuration.
- The function updates MRx, MCR, and EMR registers according to the configuration.
- Call this function after initializing the timer to set up match behavior.

H `lpc17xx_timer.h`

```
void TIM_UpdateMatchValue(
    LPC_TIM_TypeDef* TIMx,
    TIM_MATCH_CHANNEL_OPT matchChannel,
    uint32_t matchValue)
```

Updates the match value for the specified Timer/Counter channel.

This function sets the match register (MR0-MR3) of the given timer peripheral to the provided value for the selected match channel. It does not modify any match control or interrupt settings.

Params: *[in]* **TIMx** — Pointer to the timer peripheral (LPC_TIMx [0...3]).
[in] **matchChannel** — Match channel to update (TIM_MATCH_CHANNEL_x [0..3]).
[in] **matchValue** — New value to set in the match register.

Remarks:

- Only the match value is updated; match behavior must be configured separately.
- Call this function to change the match value during runtime.

H `lpc17xx_timer.h`

```
void TIM_SetMatchExt(
    LPC_TIM_TypeDef* TIMx,
    TIM_MATCH_CHANNEL_OPT matchChannel,
    TIM_EXTMATCH_OPT extMatchOutputType)
```

Sets the external match output type for a specific match channel.

This function configures the external match output behavior for the selected match channel (MAT0...MAT3) of the specified Timer/Counter peripheral. It updates the EMR register to set the output type for the given channel.

Params: *[in]* **TIMx** — Pointer to the timer peripheral (LPC_TIMx [0...3]).
[in] **matchChannel** — Match channel to configure (TIM_MATCH_CHANNEL_x [0..3]).
[in] **extMatchOutputType** — External match output type:

- TIM_NOTHING
- TIM_LOW
- TIM_HIGH
- TIM_TOGGLE

Remarks:

- Only the specified channel is affected.
- Call this function after initializing the timer and before starting it.

H `lpc17xx_timer.h`

```
void TIM_ConfigCapture(
    LPC_TIM_TypeDef* TIMx,
    TIM_CAPTURECFG_Type* TIM_CaptureConfigStruct)
```

Configures the capture channel for the specified Timer/Counter peripheral.

This function sets up the capture behavior for the selected channel, including edge detection (rising, falling), interrupt generation, and channel selection. It updates the CCR register according to the configuration structure.

Params: *[in]* **TIMx** — Pointer to the timer peripheral (LPC_TIMx [0...3]).
[in] **TIM_CaptureConfigStruct** — Pointer to a TIM_CAPTURECFG_Type.

Remarks:

- Only the specified channel is affected.
- Call this function after initializing the timer to set up capture behavior.

H `lpc17xx_timer.h`

```
void TIM_Cmd(
    LPC_TIM_TypeDef* TIMx,
    FunctionalState newState)
```

Enables or disables the specified Timer/Counter peripheral.

This function sets or clears the enable bit in the TCR register of the given timer, effectively starting or stopping the timer/counter.

Params: *[in]* **TIMx** — Pointer to the timer peripheral (LPC_TIMx [0...3]).
[in] **newState** — Functional state:

- ENABLE : Start the timer/counter.
- DISABLE : Stop the timer/counter.

Remarks:

- Use this function to control timer operation after configuration.
- The timer must be initialized before calling this function.

H `lpc17xx_timer.h`

```
uint32_t TIM_GetCaptureValue(
    LPC_TIM_TypeDef* TIMx,
    TIM_CAPTURE_CHANNEL_OPT captureChannel)
```

Reads the value of the capture register for the specified channel.

This function returns the value stored in the capture register (CR0 or CR1) of the given timer peripheral, depending on the selected capture channel.

Params: *[in]* **TIMx** — Pointer to the timer/counter peripheral (LPC_TIMx [0...3]).
[in] **captureChannel** — Capture channel to read:

- TIM_CAPTURE_CHANNEL_0 : CAPn.0 input pin for TIMERn
- TIM_CAPTURE_CHANNEL_1 : CAPn.1 input pin for TIMERn

Returns: Value of the selected capture register.

Remarks:

- Use this function to obtain the timestamp captured on the specified input.
- The timer must be configured for capture mode before using this function.

H `lpc17xx_timer.h`

```
void TIM_ResetCounter(LPC_TIM_TypeDef* TIMx)
```

Resets the Timer/Counter peripheral.

This function synchronously resets the Timer Counter (TC) and Prescale Counter (PC) of the specified timer by setting and then clearing the reset bit in the TCR register.

Params: *[in]* **TIMx** — Pointer to the timer peripheral (LPC_TIMx [0...3]).

Remarks:

- Use this function to reset the timer counters to zero.

H `lpc17xx_timer.h`